

1. Zadatak – treniranje gotove CNN mreže (ResNet18) na MNIST

Cilj

Napisati **potpunu PyTorch skriptu od nule** koja:

1. učitava MNIST podatke
2. priprema podatke za ResNet18
3. učitava gotovu ResNet18 mrežu s unaprijed naučenim težinama
4. trenira model i validira ga
5. sprema model na disk
6. logira metrike u TensorBoard

Napomena: Ne dobivate gotovu skriptu — implementaciju pišete samostalno prateći prezentaciju i objašnjenja na vježbama.

Zahtjevi implementacije

1) Instalacija i provjera okruženja

- provjerite radi li CUDA (ako imate NVIDIA GPU)
- ispišite koji se device koristi (CPU ili CUDA)

2) Dataset i DataLoader

- učitajte MNIST dataset (torchvision.datasets)
- napravite train/validation split (npr. 90% / 10%)
- napravite DataLoader za train i validation skup

3) Vizualizacija podataka

- Učitajte nekoliko uzoraka iz train skupa
- Prikažite ih pomoću matplotlib
- Provjerite:
 - kako izgleda znamenka?
 - je li pozadina crna ili bijela?

4) Transformacije (MNIST → ResNet ulaz)

Budući da ResNet18 očekuje ulaz oblika [3, 224, 224], a MNIST slike su grayscale [1, 28, 28], potrebno je koristiti točno sljedeće transformacije:

```
transform = transforms.Compose([  
    transforms.Resize((224, 224)),  
    transforms.Grayscale(num_output_channels=3),  
    transforms.ToTensor(),  
])
```

- `Resize((224, 224))` → prilagodba rezolucije za ResNet18
- `Grayscale(num_output_channels=3)` → pretvaranje grayscale slike u 3-kanalni format
- `ToTensor()` → pretvaranje u PyTorch Tensor (vrijednosti u rasponu [0, 1])

Transformaciju zatim proslijedite u MNIST dataset konstruktor (`transform=transform`).

5) Učitavanje gotovog modela

- učitajte ResNet18 koristeći moderni weights API (`ResNet18_Weights.DEFAULT`)
- prilagodite model za MNIST tako da predviđa 10 klasa (zamjena zadnjeg sloja)

Opcionalno (preporučeno):

- “zamrznite” backbone (trenira se samo zadnji sloj) radi bržeg treniranja

6) Trening i validacija

Implementirajte:

- trening petlju
 - `model.train()`
 - `optimizer.zero_grad()`
 - `forward` → `loss` → `backward` → `step`
- validacijsku petlju
 - `model.eval()`
 - `torch.no_grad()`

Na kraju svake epohe ispišite:

- `train loss` i `train accuracy`
- `validation loss` i `validation accuracy`

7) Spremanje modela

- spremite model na disk koristeći `state_dict` (npr. `resnet18_mnist.pth`)

8) TensorBoard logiranje

Dodajte TensorBoard logiranje:

- kreirajte SummaryWriter
- logirajte train/val loss i accuracy
- pokrenite TensorBoard i provjerite da se grafovi prikazuju

Očekivani rezultat

- Skripta se može pokrenuti bez greške
- Tijekom treniranja se vide metrike u terminalu
- TensorBoard prikazuje grafove loss/accuracy
- Model se uspješno sprema na disk

2. Zadatak - Učitavanje modela i inferencija na vlastitoj slici

Cilj

Učitati prethodno spremljeni model (resnet18_mnist.pth) i napraviti predikciju nad slikom znamenke koju sami nacrtate.

Opis zadatka

U ovom zadatku:

1. učitati ćete model s diska
2. nacrtati ćete znamenku (0–9) u Paintu
3. prilagoditi sliku tako da bude kompatibilna s modelom
4. pokrenuti inferenciju
5. ispisati predviđenu klasu

1) Učitavanje modela

- kreirajte istu arhitekturu kao u prvom zadatku (ResNet18)
- prilagodite zadnji sloj na 10 klasa
- učitajte težine pomoću load_state_dict
- postavite model u eval() način rada

2) Izrada vlastite slike

U Paintu (ili drugom alatu):

- nacrtajte jednu znamenku (0–9)
- koristite bijelu znamenku na crnoj pozadini
- spremite sliku kao PNG ili JPG
- preporučena rezolucija: 28×28 (nije obavezno)

3) Priprema slike za model

Slika mora proći **iste transformacije kao tijekom treniranja:**

```
transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.Grayscale(num_output_channels=3),
    transforms.ToTensor(),
])
```

Važno:

- redoslijed transformacija mora biti isti
- broj kanala mora biti 3
- oblik mora biti [1, 3, 224, 224] (dodati batch dimenziju)

4) Inferencija

- koristite `model.eval()`
- koristite `torch.no_grad()`
- proslijedite sliku modelu
- koristite `argmax` za dobivanje predviđene klase

Ispišite:

Predviđena znamenka: X

Očekivani rezultat

- Model se uspješno učitava bez greške
- Slika se pravilno transformira
- Inferencija vraća jednu klasu (0–9)
- Program ispisuje predikciju